

PARADIGMAS DE LA PROGRAMACIÓN

Tarea 2

Desarrollo de un Sistema de Gestión de Rutinas Deportivas mediante Java Swing y Programación Orientada a Eventos



**Universidad
Andrés Bello**

Autor:

Christopher Torrejón ^(a, b)

^(a) Ingeniería Civil Industrial. Facultad de Ingeniería, Universidad Andrés Bello. Chile

^(b) Ingeniería Civil Informática. Facultad de Ingeniería, Universidad Andrés Bello. Chile

c.torrejnallende@uandresbello.edu

21.774.845-3

Paradigmas de la Programación **[8025]**

Profesor encargado:

Juan Felipe Calderón Maureira

Mayo 28, 2026. Valparaíso, Viña del Mar

Tabla de contenido

1.	Introducción	1
1.1.	Motivación.....	1
1.2.	Objetivo	1
1.2.1.	Objetivo General.....	1
1.2.2.	Objetivos Específicos	2
1.3.	Estructura del Documento	2
2.	Descripción de la Solución.....	3
2.1.	Modelo Propuesto.....	3
2.2.	Clases Diseñadas.....	3
2.2.1.	Figura 1. Diagrama de Clases.....	5
2.3.	Relaciones entre Componentes del Sistema	6
2.4.	Justificación	6
3.	Conclusión	8
3.1.	Reflexión	8
3.2.	Análisis de resultados y objetivos.....	8
3.2.1.	Resolución del Objetivo General	8
3.2.2.	Objetivo específico 1	9
3.2.3.	Objetivo específico 2	9
3.2.4.	Objetivo específico 3	9
4.	Referencias	10

1. Introducción

1.1. Motivación

El desarrollo de la presente tarea surge como una evolución natural del trabajo realizado previamente en la primera etapa del proyecto, implementada en lenguaje C++. En dicha instancia, el objetivo principal consistió en modelar correctamente los datos asociados a un sistema de gestión de rutinas de entrenamiento personalizadas, además de implementar las restricciones y funcionalidades solicitadas, tales como la creación de ejercicios, generación de rutinas y consulta de información relevante.

Si bien aquella solución permitió aplicar fundamentos sólidos de programación orientada a objetos, el sistema se encontraba limitado a un entorno de consola, lo que dificultaba la interacción con el usuario y restringía considerablemente la experiencia de uso. Bajo este contexto, la segunda etapa del proyecto incorporó nuevos desafíos relacionados con el desarrollo de interfaces gráficas, persistencia de información y comunicación entre distintos componentes del sistema mediante una arquitectura separada en frontend y backend.

La principal motivación detrás de este trabajo radica en la transición desde aplicaciones tradicionales de consola hacia sistemas más cercanos a entornos reales de software, en donde la interacción gráfica, la organización modular y el tratamiento estructurado de los datos adquieren un rol fundamental. En este sentido, la utilización de Java Swing permitió construir una interfaz visual capaz de interactuar dinámicamente con la lógica de negocio, facilitando tanto la navegación como la validación de información ingresada por el usuario.

Asimismo, el proyecto permitió comprender de manera práctica la importancia de separar responsabilidades dentro de un sistema computacional. La división entre frontend y backend favoreció la organización del código, el mantenimiento del sistema y la protección de la integridad de los datos, permitiendo que cada componente desempeñara funciones específicas sin comprometer el funcionamiento del resto de la aplicación. Esta separación también introdujo conceptos asociados a la comunicación entre módulos, programación orientada a eventos y diseño modular, elementos ampliamente utilizados en el desarrollo moderno de software.

Finalmente, el desarrollo de esta solución representó una instancia significativa de aprendizaje, ya que permitió integrar conocimientos de programación orientada a objetos, estructuras de datos, validación de información, gestión de eventos e interfaces gráficas, aproximando el proceso de desarrollo a escenarios más representativos de aplicaciones reales.

1.2. Objetivo

1.2.1. Objetivo General

- Implementar un sistema de gestión de rutinas deportivas en lenguaje Java, trasladando la lógica previamente desarrollada en C++ e incorporando una interfaz gráfica basada en Java Swing mediante programación orientada a eventos, con el propósito de fortalecer la interacción usuario-sistema y la organización modular de la aplicación.

1.2.2. Objetivos Específicos

1. Aplicar los principios de programación orientada a objetos y programación orientada a eventos en Java, trasladando la lógica previamente desarrollada en C++ hacia una solución gráfica basada en Java Swing.
2. Diseñar una arquitectura modular separada en frontend y backend, asegurando una correcta comunicación entre componentes y una gestión consistente de la información dentro del sistema.
3. Implementar mecanismos de validación y control de datos que permitan detectar errores de ingreso por parte del usuario, fortaleciendo la integridad de la información y la interacción con la aplicación.

1.3. Estructura del Documento

El presente informe se encuentra estructurado en distintas secciones orientadas a describir de manera progresiva el desarrollo de la solución implementada. Inicialmente, se presenta una introducción compuesta por la motivación del proyecto, el objetivo general y los objetivos específicos, permitiendo contextualizar la problemática abordada y los principales propósitos del trabajo realizado.

Posteriormente, se desarrolla la sección correspondiente a la descripción de la solución, en donde se exponen los principales componentes del sistema, incluyendo el diseño de clases, la arquitectura implementada y las relaciones existentes entre los distintos módulos que conforman la aplicación. Dentro de esta sección, se incorpora además el diagrama de clases general del sistema, el cual representa gráficamente la organización estructural de la solución desarrollada.

Asimismo, el presente informe se complementa con una ficha técnica asociada al apartado “2.2. Clases Diseñadas”, cuyo propósito es profundizar en la lógica interna del programa. En dicho documento se describen detalladamente las responsabilidades de cada clase, las dependencias existentes entre ellas y el fundamento técnico asociado a su implementación, proporcionando una visión más específica sobre el funcionamiento interno de la aplicación y las decisiones de diseño adoptadas durante el desarrollo.

Finalmente, el documento concluye con una sección de conclusiones, en donde se realiza una reflexión respecto al proceso de desarrollo y al cumplimiento de los objetivos planteados, además de incorporar las referencias bibliográficas utilizadas como apoyo durante la elaboración del proyecto.

2. Descripción de la Solución

2.1. Modelo Propuesto

El sistema desarrollado tiene como finalidad gestionar la creación y administración de rutinas de entrenamiento para los clientes de un centro deportivo. En este contexto, los principales usuarios del programa corresponden a los trabajadores o administradores del establecimiento, quienes interactúan con el sistema mediante una interfaz gráfica diseñada para facilitar la navegación y el acceso a las distintas funcionalidades implementadas.

La solución fue desarrollada en lenguaje Java utilizando la biblioteca Java Swing (Oracle, n.d.), incorporando principios de programación orientada a objetos y programación orientada a eventos. Esto permitió representar computacionalmente tanto los ejercicios como las rutinas del sistema, además de gestionar de manera dinámica la interacción entre el usuario y la interfaz gráfica mediante eventos asociados a botones, ventanas y componentes visuales.

A nivel estructural, el programa se encuentra dividido en dos bloques principales: frontend y backend. El frontend concentra todas las ventanas y componentes gráficos visibles para el usuario, siendo responsable de la interacción visual y de la captura de acciones realizadas dentro de la aplicación. Por otra parte, el backend contiene la lógica de negocio, la estructura de datos y las reglas de funcionamiento del sistema, incluyendo tanto los requerimientos explícitamente establecidos en el enunciado como restricciones adicionales incorporadas para preservar la coherencia lógica de la aplicación.

Dentro de este último bloque, la clase *Orquestador* cumple un rol central, actuando como coordinador principal entre las distintas estructuras del sistema. Su función consiste en controlar el flujo lógico del programa, validar restricciones y asegurar que las rutinas sean generadas correctamente según los requisitos establecidos. Asimismo, esta clase permite incorporar validaciones adicionales relacionadas con reglas de negocio implícitas, como evitar la creación duplicada de rutinas para un mismo cliente dentro de una misma semana, preservando así la integridad de la información almacenada.

Finalmente, el flujo de funcionamiento del programa se basa en una metodología cíclica e interactiva, donde el usuario navega entre distintas ventanas según las funcionalidades que requiera utilizar. Para ello, resulta fundamental la correcta carga de datos iniciales, ya que gran parte de las operaciones del sistema dependen de la información previamente ingresada desde archivos externos.

2.2. Clases Diseñadas

Con el propósito de estructurar adecuadamente las responsabilidades del sistema, las clases desarrolladas fueron organizadas en dos bloques principales: backend y frontend (Microsoft, n.d.). Esta separación permite distinguir claramente entre la lógica de negocio del programa y la interacción gráfica con el usuario, favoreciendo la modularidad y el mantenimiento de la aplicación.

Backend

- Ejercicio
- EjercicioCardiovascular extends Ejercicio
- EjercicioFuerza extends Ejercicio
- ColeccionEjercicio
- Rutina
- ColeccionRutina
- Orquestador
- ProcesamientoDatos
- LecturaArchivo

Frontend

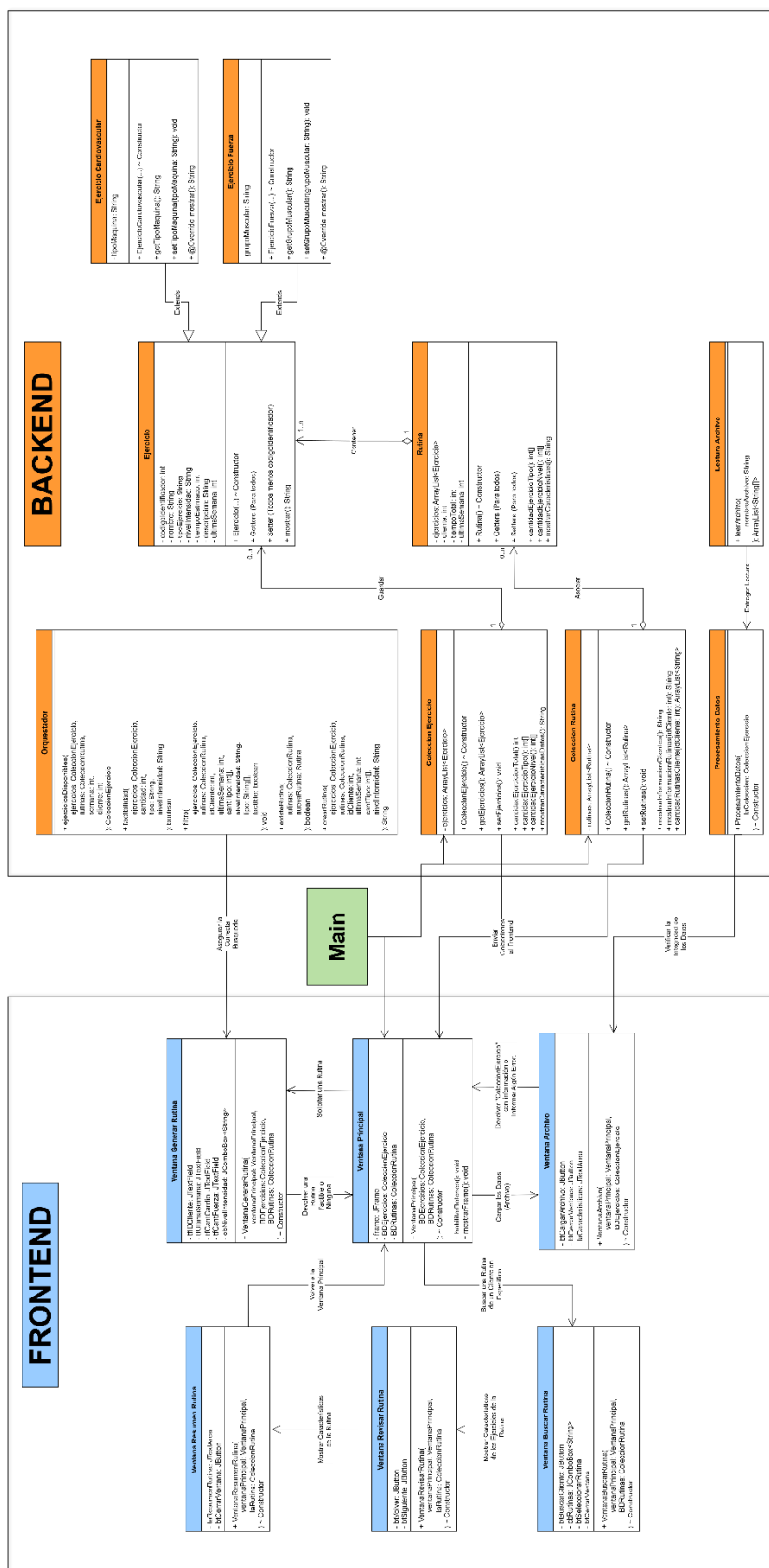
- VentanaPrincipal
- VentanaArchivo
- VentanaGenerarRutina
- VentanaBuscarRutina
- VentanaRevisarRutina
- VentanaResumenRutina

Dentro de la arquitectura implementada, algunas clases cumplen además funciones asociadas a controladores lógicos del sistema. Entre ellas destaca principalmente la clase *Orquestador*, encargada de coordinar la comunicación entre los distintos componentes del backend y controlar el flujo principal de generación y validación de rutinas. Esta clase centraliza gran parte de la lógica de negocio, permitiendo aplicar tanto las restricciones definidas explícitamente en el enunciado como reglas adicionales orientadas a preservar la coherencia del sistema.

De manera complementaria, la clase *ProcesamientoDatos* también desempeña un rol similar al de un controlador, ya que se encarga de gestionar el tratamiento y validación de la información obtenida desde archivos externos antes de integrarla a las estructuras principales del programa. Esto permite mantener separadas las responsabilidades asociadas al procesamiento de datos respecto de la lógica principal de la aplicación.

Finalmente, en la **Figura 1** se presenta el diagrama de clases UML (OMG, 2024) correspondiente a la solución desarrollada, el cual representa la estructura general del sistema y las relaciones existentes entre sus distintos componentes. Asimismo, para obtener una descripción más detallada sobre el funcionamiento técnico y lógico de cada clase, se incorpora una ficha técnica complementaria en el repositorio del presente proyecto.

2.2.1. Figura 1. Diagrama de Clases



Tecnologías:

- Java
- Java Swing
- Programación Orientada a Objetos
- Arquitectura Frontend/Backend

Frontend:	Interacción con el usuario y navegación.
Backend:	Lógica de negocio y manejo de datos.

- △ Herencia
- ◇ Asociación / Contención
 - Dependencia / Flujo
 - 0..n Cardinalidad múltiple
 - 1..n Relación obligatoria

2.3. Relaciones entre Componentes del Sistema

La arquitectura implementada dentro del sistema se encuentra basada en una separación entre frontend y backend, permitiendo distribuir adecuadamente las responsabilidades asociadas tanto a la interacción gráfica como al procesamiento lógico de la aplicación. Esta división busca evitar que la lógica principal del sistema y las estructuras de datos queden expuestas directamente a la interfaz visual, favoreciendo así una mejor organización, mantenimiento y protección de la información utilizada durante la ejecución del programa.

De manera conceptual, el frontend puede entenderse como la capa visible del sistema, es decir, el conjunto de ventanas y componentes gráficos con los cuales interactúa el usuario. Por otra parte, el backend representa el núcleo interno de la aplicación, donde se encuentran implementadas las estructuras de datos, reglas de negocio y mecanismos de validación que permiten garantizar el correcto funcionamiento del sistema. Bajo esta lógica, la interacción entre ambos bloques requiere una comunicación coherente y persistente, permitiendo que las solicitudes realizadas desde la interfaz gráfica sean procesadas adecuadamente por la lógica interna del programa.

Dentro del frontend, las acciones realizadas por el usuario son gestionadas mediante programación orientada a eventos. Esto implica que eventos asociados a componentes gráficos (como presionar botones, ingresar información o navegar entre ventanas) generan solicitudes que posteriormente son atendidas por el backend. Gracias a esta metodología, el sistema puede responder dinámicamente a las acciones realizadas por el usuario, manteniendo una interacción fluida y organizada entre la interfaz gráfica y la lógica de negocio.

Finalmente, la representación estructural de la información se encuentra distribuida entre las distintas clases del backend, las cuales modelan entidades reales del sistema, tales como ejercicios, tipos de ejercicios y rutinas deportivas. Gracias a esta organización modular, el programa logra mantener una estructura coherente entre representación visual, procesamiento lógico y administración de datos.

2.4. Justificación

El desarrollo de la solución requirió la incorporación de diversas reglas de negocio orientadas a preservar la coherencia lógica del sistema y garantizar una correcta gestión de las rutinas deportivas generadas para los clientes del centro deportivo. Estas restricciones fueron implementadas tanto a partir de los requerimientos explícitos establecidos en el enunciado como mediante validaciones adicionales incorporadas durante el proceso de desarrollo.

Una de las principales restricciones implementadas corresponde a la imposibilidad de asignar los mismos ejercicios a un mismo cliente durante dos semanas consecutivas. Esta decisión busca evitar la repetición inmediata de rutinas, promoviendo una mayor variabilidad en los ejercicios asignados y obligando al sistema a discriminar automáticamente aquellos ejercicios que hayan sido utilizados durante la semana anterior del mismo cliente.

Como consecuencia de esta lógica, el sistema debe verificar previamente si existe una cantidad suficiente de ejercicios disponibles que permitan generar una nueva rutina según los parámetros solicitados por el usuario. Para ello, se consideran factores como la cantidad de ejercicios requeridos por tipo y el nivel de intensidad asociado a la rutina, asegurando que únicamente se generen rutinas válidas y coherentes con las restricciones definidas.

Asimismo, se incorporó una validación adicional orientada a impedir la creación de rutinas duplicadas para un mismo cliente dentro de una misma semana. Si bien esta restricción no se encontraba explícitamente definida dentro del enunciado original, su implementación fue considerada necesaria para preservar la consistencia lógica del sistema y evitar redundancia en la información almacenada.

Por otra parte, la incorporación de la ventana “Buscar Rutina” responde a la necesidad de gestionar adecuadamente múltiples clientes y múltiples rutinas dentro del sistema. Esta funcionalidad permite localizar específicamente la rutina deseada mediante criterios asociados al cliente y la semana correspondiente, facilitando la navegación y organización de la información almacenada por la aplicación.

Adicionalmente, se implementaron mecanismos de validación asociados a la carga de archivos externos, permitiendo verificar que los datos ingresados sean compatibles con el formato esperado por el sistema. En caso de detectar inconsistencias, errores de lectura o estructuras inválidas, la aplicación notifica adecuadamente al usuario, evitando que información incorrecta sea incorporada a las colecciones principales del programa.

Finalmente, también se consideró la validación de los datos ingresados directamente por el usuario dentro de la interfaz gráfica. Esto incluye controles asociados al ingreso de valores numéricos válidos, restricciones sobre datos negativos y verificación general de formatos esperados, permitiendo reducir errores durante la interacción con el sistema y fortaleciendo la integridad de la información procesada por la aplicación.

3. Conclusión

3.1. Reflexión

El desarrollo del presente proyecto permitió implementar exitosamente un sistema de gestión de rutinas deportivas capaz de satisfacer los requerimientos establecidos tanto a nivel funcional como estructural. La transición desde una solución previamente desarrollada en C++ hacia una aplicación construida en Java con interfaz gráfica representó un desafío significativo, especialmente debido a la incorporación de programación orientada a eventos y a la necesidad de coordinar correctamente la interacción entre frontend y backend.

A lo largo del desarrollo, fue posible aplicar principios de programación orientada a objetos, modularidad y separación de responsabilidades, permitiendo construir una solución organizada y coherente desde el punto de vista arquitectónico. La utilización de una estructura dividida entre frontend y backend favoreció la distribución lógica de tareas dentro del sistema, mientras que la implementación de clases controladoras permitió centralizar adecuadamente las validaciones y reglas de negocio necesarias para preservar la integridad de la información.

Asimismo, el sistema desarrollado logró incorporar mecanismos de validación orientados tanto al procesamiento de archivos como a la interacción directa con el usuario, permitiendo detectar errores de ingreso y situaciones inconsistentes antes de que afectaran el funcionamiento interno de la aplicación. Gracias a ello, el programa presenta un comportamiento suficientemente robusto para responder adecuadamente a distintos escenarios de uso, considerando restricciones asociadas a ejercicios repetidos, generación de rutinas y manejo de múltiples clientes.

Por otra parte, la implementación de interfaces gráficas mediante Java Swing permitió abandonar el enfoque tradicional basado exclusivamente en aplicaciones de consola, aproximando el desarrollo realizado a entornos más representativos de aplicaciones reales. Esto no solo fortaleció la experiencia de usuario, sino que también permitió comprender de manera práctica la importancia de la comunicación entre componentes visuales y lógicos dentro de un sistema computacional.

Finalmente, el proyecto representó una experiencia significativa de aprendizaje, ya que permitió integrar conocimientos relacionados con programación orientada a objetos, programación orientada a eventos, validación de datos, arquitectura modular y diseño de interfaces gráficas. Como resultado, se obtuvo una aplicación funcional, estructurada y coherente con los objetivos planteados inicialmente, logrando satisfacer las necesidades del enunciado y proporcionando una solución capaz de responder de manera adecuada a las distintas acciones y requerimientos del usuario.

3.2. Análisis de resultados y objetivos

3.2.1. Resolución del Objetivo General

El objetivo general del proyecto fue cumplido satisfactoriamente mediante la implementación de un sistema de gestión de rutinas deportivas desarrollado en lenguaje Java, incorporando una interfaz gráfica basada en Java Swing y aplicando principios de programación orientada a eventos. Asimismo, se logró trasladar correctamente la lógica previamente desarrollada en C++, manteniendo las funcionalidades y restricciones establecidas durante la primera etapa del proyecto.

La solución obtenida permitió integrar exitosamente la interacción gráfica con la lógica de negocio, favoreciendo una experiencia de usuario más dinámica y cercana a aplicaciones reales. Además, la estructura modular implementada permitió separar adecuadamente las responsabilidades asociadas al frontend y backend, contribuyendo a una organización coherente y mantenible del sistema.

3.2.2. Objetivo específico 1

El primer objetivo específico, relacionado con la aplicación de programación orientada a objetos y programación orientada a eventos en Java, fue cumplido mediante el desarrollo de un sistema estructurado a partir de clases, herencia y encapsulamiento, además de la utilización de eventos asociados a componentes gráficos de Java Swing.

La lógica desarrollada originalmente en C++ pudo ser adaptada correctamente al entorno Java, permitiendo mantener las funcionalidades principales del sistema e incorporando adicionalmente mecanismos de interacción gráfica basados en listeners y acciones generadas por el usuario dentro de las distintas ventanas de la aplicación.

3.2.3. Objetivo específico 2

El segundo objetivo específico, orientado al diseño de una arquitectura modular basada en frontend y backend, fue alcanzado mediante la separación clara entre la interfaz gráfica y la lógica de negocio del sistema.

Esta organización permitió distribuir adecuadamente las responsabilidades entre componentes visuales y estructuras lógicas, evitando el acoplamiento excesivo entre clases y favoreciendo una comunicación coherente entre ambos bloques del programa. Asimismo, la incorporación de clases controladoras permitió centralizar gran parte de las validaciones y restricciones necesarias para garantizar la consistencia de la información procesada por la aplicación.

3.2.4. Objetivo específico 3

El tercer objetivo específico, relacionado con la implementación de mecanismos de validación y control de datos, también fue cumplido satisfactoriamente mediante la incorporación de distintas verificaciones orientadas tanto a la carga de archivos como al ingreso de información por parte del usuario.

El sistema desarrollado permite detectar errores asociados a formatos inválidos, valores inconsistentes y datos incompatibles con las restricciones definidas por el programa. De igual manera, se implementaron validaciones adicionales relacionadas con reglas de negocio, tales como evitar la repetición de ejercicios en semanas consecutivas y prevenir la creación duplicada de rutinas para un mismo cliente dentro de una misma semana.

Gracias a estas medidas, la aplicación logró mantener un comportamiento coherente y suficientemente robusto frente a distintos escenarios de uso, fortaleciendo la integridad de la información y mejorando la experiencia general del usuario durante la interacción con el sistema.

4. Referencias

Microsoft. (s.f.). *learn*. Obtenido de <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/>

OMG, G. O. (2024). *Unified Modeling Language (UML)*. Obtenido de <https://www.omg.org/spec/UML/>

Oracle. (s.f.). *Java Documentation*. Obtenido de <https://docs.oracle.com/javase/tutorial/uiswing/>